Advanced Web Application Development Technologies

Nikolaos Korfiatis & Dimitris Ringas

nkorf@ionio.gr | riggas@ionio.gr

Department of Informatics

Ionian University



HTTP and Python

Web Technologies & API Calls



Session Objectives

- **Understanding** the operation and importance of the HTTP protocol
- **Familiarization** with HTTP methods, status codes and headers
- **Practical application** of HTTP calls with Python
- **Handling** errors and timeouts
- **Processing** JSON payloads in requests and responses
- **Developing** skills for consuming REST APIs

What is HTTP?

The **HyperText Transfer Protocol (HTTP)** is the fundamental protocol of the World Wide Web:

- **Application Layer Protocol** in the TCP/IP stack
- **Client-Server Architecture** - request and response
- **Stateless Protocol** - each request is independent
- **Text-based** - human readable format
- **Port 80** (HTTP) and **Port 443** (HTTPS)

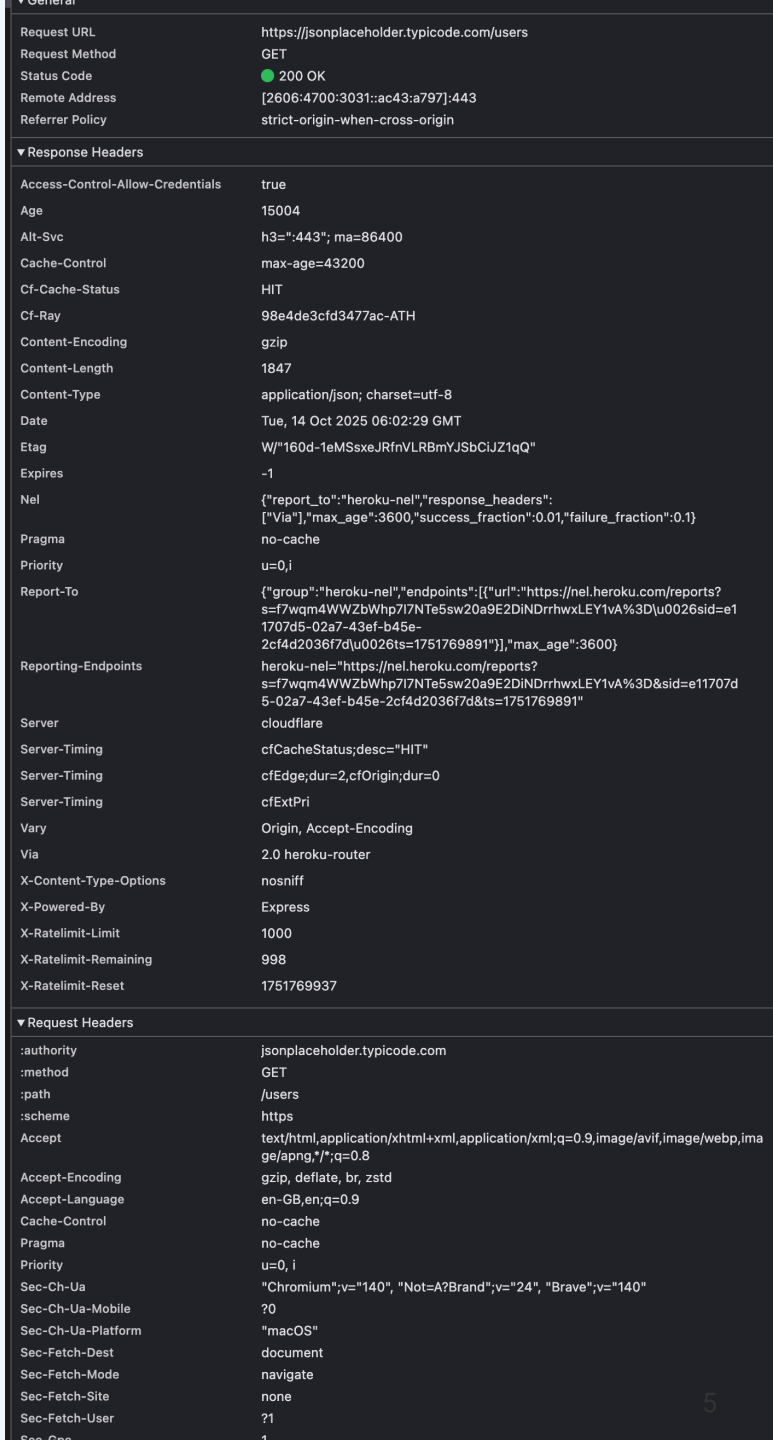
Why is it important?

- Basis of all web applications
- Foundation for REST APIs
- Communication between microservices
- Data exchange in distributed systems

HTTP Request Structure

Components of an HTTP Request:

1. **Request Line**: Method + URL + HTTP Version
2. **Headers**: Metadata about the request
3. **Body**: Data (optional, depending on method)



▾ General

Request URL	https://jsonplaceholder.typicode.com/users
Request Method	GET
Status Code	200 OK
Remote Address	[2606:4700:3031::ac43:a797]:443
Referrer Policy	strict-origin-when-cross-origin

▾ Response Headers

Access-Control-Allow-Credentials	true
Age	15004
Alt-Svc	h3=":443"; ma=86400
Cache-Control	max-age=43200
Cf-Cache-Status	HIT
Cf-Ray	98e4de3cfd3477ac-ATH
Content-Encoding	gzip
Content-Length	1847
Content-Type	application/json; charset=utf-8
Date	Tue, 14 Oct 2025 06:02:29 GMT
Etag	W/"160d-1eMSsxeJRfnVLRBmYJSbCiJZ1qQ"
Expires	-1
Nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction":0.01,"failure_fraction":0.1}
Pragma	no-cache
Priority	u=0,i
Report-To	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=f7wqm4WWZbWhp7I7NTe5sw20a9E2DiNDrrhwxLEY1vA%3D\u0026sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d\u0026ts=1751769891"}],"max_age":3600}
Reporting-Endpoints	heroku-nel="https://nel.heroku.com/reports?s=f7wqm4WWZbWhp7I7NTe5sw20a9E2DiNDrrhwxLEY1vA%3D&sid=e11707d5-02a7-43ef-b45e-2cf4d2036f7d&ts=1751769891"
Server	cloudflare
Server-Timing	cfCacheStatus;desc="HIT"
Server-Timing	cfEdge;dur=2,cfOrigin;dur=0
Server-Timing	cfExtPri
Vary	Origin, Accept-Encoding
Via	2.0 heroku-router
X-Content-Type-Options	nosniff
X-Powered-By	Express
X-Ratelimit-Limit	1000
X-Ratelimit-Remaining	998
X-Ratelimit-Reset	1751769937

▾ Request Headers

:authority	jsonplaceholder.typicode.com
:method	GET
:path	/users
:scheme	https
Accept	text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-GB,en;q=0.9
Cache-Control	no-cache
Pragma	no-cache
Priority	u=0, i
Sec-Ch-Ua	"Chromium";v="140", "Not=A?Brand";v="24", "Brave";v="140"
Sec-Ch-Ua-Mobile	?0
Sec-Ch-Ua-Platform	"macOS"
Sec-Fetch-Dest	document
Sec-Fetch-Mode	navigate
Sec-Fetch-Site	none
Sec-Fetch-User	?1

HTTP Methods (Verbs)

Method	Purpose	Body	Idempotent*
GET	Retrieve data	No	Yes
POST	Create new	Yes	No
PUT	Full update	Yes	Yes
PATCH	Partial update	Yes	No
DELETE	Delete	No	Yes
HEAD	Headers only	No	Yes
OPTIONS	Allowed methods	No	Yes

Mapping to CRUD Operations:

- **Create** → POST
- **Read** → GET
- **Update** → PUT/PATCH
- **Delete** → DELETE

“ * An HTTP method is idempotent when executing it repeatedly has the same effect as executing it once.
⇒ If we send the same HTTP request multiple times, the server state does not change any further after the first call. ”

HTTP Status Codes

1xx Informational

- `100 Continue` - Continue the request

2xx Success

- `200 OK` - Successful request
- `201 Created` - New resource created
- `204 No Content` - Success with no content

3xx Redirection

- `301 Moved Permanently` - Permanent move
- `302 Found` - Temporary move

Example Response:



HTTP Status Codes (continued)

5xx Server Error

- `500 Internal Server Error` - Generic server error
- `502 Bad Gateway` - Problem at the gateway
- `503 Service Unavailable` - Service not available
- `504 Gateway Timeout` - Timeout at the gateway

4xx Client Error

- `400 Bad Request` - Wrong syntax
- `401 Unauthorized` - Authentication required
- `403 Forbidden` - Access not allowed
- `404 Not Found` - Resource not found
- `429 Too Many Requests` - Rate limiting

HTTP Headers - Main Categories

Request Headers

- `Host: api.example.com` - Server hostname
- `User-Agent: MyApp/1.0` - Client information
- `Accept: application/json` - Preferred content types
- `Authorization: Bearer token123` - Authentication
- `Content-Type: application/json` - Body format

Response Headers

- `Content-Type: application/json` - Response format
- `Content-Length: 256` - Body size in bytes
- `Server: nginx/1.18.0` - Server software
- `Cache-Control: max-age=3600` - Caching instructions

Custom Headers

- `X-API-Key: your-api-key`
- `X-Request-ID: uuid-here`
- `X-Rate-Limit-Remaining: 99`

Python and HTTP



Python and HTTP - Introduction

Alternative options for HTTP calls in python:

Built-in Libraries

- `urllib.request` - Standard library
- `http.client` - Low-level HTTP client

Third-party Libraries

- **`requests`** - Human-friendly HTTP library
- `httpx` - Async HTTP client
- `aiohttp` - Asynchronous HTTP client/server

Why `requests`?

- **Simple API** - easy to use
- **Feature-rich** - complete feature set
- **Well-documented** - excellent documentation
- **Community support** - large community

Example request in python:



Python Environment Setup

1. Python Installation



2. Virtual Environment Setup



3. Install Required Packages



The requests library - Basic Usage

Simple GET Request



Response Object Properties



GET Requests with Parameters

Query Parameters



Headers Customization



POST Requests and Data Sending

Sending JSON Data



Form Data



Error Handling and Exceptions

HTTP Error Handling



Timeouts and Performance

Timeout Configuration



Timeouts and Performance

Session Objects for Performance



- When we use plain requests.get() or requests.post() calls, each call performs the following steps:
  - Opens a new TCP connection to the server.
  - Performs a TLS handshake (if HTTPS).
  - Sends the request.
  - Receives the response.
  - Closes the connection.
- When we use a Session object, we tell Python to reuse the same TCP connection across requests:
  - A pool of active connections is created for each host.
  - Connections are not closed after each request, but kept alive.
  - Cookies and headers remain stored in the session.
  - Requests are much faster, since repeated connection setup is avoided.

JSON Processing with Python

JSON Parsing



Nested JSON Navigation



✋ That's enough.. *for today*!

Continue at:

https://github.com/riggas-ionio/web-II/

